

Bloque I: Concurrencia

Práctica 1: Contadores

Asignatura: Concurrencia y Paralelismo **Fecha Límite:** 13 de febrero de 2026 **Entrega:** Aula Virtual

Vamos a implementar un sistema donde varios *threads* trabajan con dos contadores: *a* y *b*, que empiezan con valor 0. Habrá dos tipos de *threads*:

- El primero suma uno en *a* y resta uno en *b*.
- El segundo resta uno en *a* y suma uno en *b*.

El número de *threads* de cada tipo, así como el número de iteraciones que hace cada *thread*, y una espera en nanosegundos que hace cada *thread* después de realizar una iteración, se configuran mediante constantes declaradas con `#define` al principio del código.

Ejercicio 1 (Threads de tipo a)

Implemente el lanzamiento de los *threads* de tipo *a*. Los *threads* de tipo *a* deberían recibir sus parámetros a través de una estructura que se crea en el *heap*. El *thread* principal debería esperar a que todos los *threads* de tipo *a* terminen.

Los contadores no pueden declararse como variables globales. Deberán pasarse a los *threads* a través de la estructura. Declárelos como variables `volatile`, para evitar que el compilador optimice las lecturas y escrituras.

Ejercicio 2 (Threads de tipo b)

Implemente el lanzamiento de los *threads* de tipo *b*. Los *threads* de tipo *b* deberían recibir sus parámetros a través de una estructura que se crea en el *stack* del *thread* principal. El *thread* principal debería esperar a que todos los *threads* de tipo *b* terminen, pero los *threads* de tipo *a* y *b* deberían ejecutarse concurrentemente.

La función donde se ejecutan los *threads* de tipo *b* debería ser la misma que los de tipo *a*, sin ningún tipo de comprobación de en qué tipo de *thread* estamos. La estructura que se pasa a los *threads* debería incluir un puntero al contador en el que sumar, y un puntero al contador en el que restar.

Ejercicio 3 (Thread de monitorización)

Implemente un *thread* de monitorización que se lance desde el *thread* principal justo antes de lanzar los *threads* de tipo *a* y *b*, y que termine justo después de que todos los *threads* de tipo *a* y *b* hayan terminado. El *thread* de monitorización debería imprimir el valor de los contadores y su suma de forma periódica.

Ejercicio 4 (Proteja los contadores con un mutex)

Proteja las operaciones sobre los contadores con un *mutex*. El *thread* de monitorización debería utilizar el *mutex*, y si está implementado de forma correcta la suma debería ser siempre 0. Ese *mutex* debería pasarse a los *threads* a través de la estructura de argumentos.

Ejercicio 5 (Separe el mutex de cada contador)

En vez de utilizar un único *mutex* para proteger ambos contadores, utilice un *mutex* para cada contador. Esos *mutex* deberán pasarse a los *threads* a través de la estructura de argumentos en dos campos: uno para el *mutex* que protege el contador en el que se suma, y otro que protege el contador en el que se resta.

Práctica 2: Librerías Mutex / Semáforos

Asignatura: Concurrency y Paralelismo

Fecha Límite: 6 de marzo de 2026

Plataforma: GitHub Classroom

Ejercicio 1 (Mutex recursivos)

Un *mutex* recursivo permite al *thread* que ha bloqueado un *mutex* volver a bloquearlo un número arbitrario de veces. El *mutex* solo queda desbloqueado cuando se ha hecho el mismo número de desbloques (*unlocks*) que de bloqueos (*locks*).

Es posible implementar *mutex* recursivos utilizando como base los de la librería `pthread`, más la función `pthread_t pthread_self()`, que devuelve el identificador del *thread* que la llama.

Implemente estos *mutex* en `rec_mutex.c`, siguiendo la interfaz definida en `rec_mutex.h`. Compruebe que funcionan correctamente escribiendo un programa de prueba en el que varios *threads* trabajan con un *array* de un cierto tamaño inicializado con los números de 0 en adelante. Cada *thread* itera generando dos posiciones aleatorias e intercambiándolas. Cada posición debe estar protegida por su propio `rec_mutex`.

Ejercicio 2 (Mutex prioridades)

Implemente, utilizando como base los *mutex* de la librería `pthread`, una librería donde se puedan bloquear *mutex* con distinta prioridad. El número de prioridades distintas se configura al crear el *mutex*, y puede asumirse que va a ser un número razonablemente pequeño.

La operación `mutex_prio_lock(mutex_t *m, int prio)` intenta bloquear el *mutex* con prioridad `prio`. El *mutex* solo podrá ser bloqueado si está libre, o si no hay ningún otro *thread* intentando hacer un bloqueo con mayor prioridad.

Diseñe e implemente un programa de prueba que compruebe el funcionamiento correcto de estos *mutex*.

Ejercicio 3 (Mutex Read/Write)

Una forma de resolver el problema de lectores/escritores es implementar un *mutex* donde se puedan hacer bloqueos para lectura (`mutex_read_lock`) y para escritura (`mutex_write_lock`). Un bloqueo de escritura obtiene acceso exclusivo a la zona compartida, pero un bloqueo de lectura es compatible con otros bloqueos de lectura.

Implemente una librería de *mutex* con bloqueos de lectura/escritura según la interfaz definida en el fichero `rw_mutex.h`.

Ejercicio 4 (Semáforos)

Vamos a implementar, utilizando los *mutex* y variables de condición de la librería `pthread`, nuestra propia librería de semáforos. Para ello, implemente la interfaz definida en el fichero `sem.h` dentro del fichero `sem.c`. En la estructura `sem_t` solo pueden usarse tipos básicos de C, *mutex* y condiciones.

Implemente el problema del barbero usando semáforos descrito en clase utilizando la librería que acabamos de definir en un fichero `barber.c`. Tanto los barberos como los clientes deberían imprimir un mensaje cuando vayan a cortar el pelo indicando su número de barbero / cliente.

Práctica 3: Paso de Mensajes (Erlang)

Asignatura: Concurrencia y Paralelismo

Fecha Límite: 20 de marzo de 2026

Plataforma: GitHub Classroom

Ejercicio 1 (Queue)

Implemente un módulo `queue` que implemente una cola en Erlang. No es necesario crear un proceso para guardar la cola. La cola debe implementarse basándose en dos listas que mantengan, respectivamente, el principio y el final de la cola. Por ejemplo, considere la cola `[1, 2, 3, 4, 5]`, donde 1 es el elemento al frente y 5 el último. Una posible representación con dos listas sería `[1, 2, 3]` y `[5, 4]`, donde el principio de la cola está en orden, y el final al revés.

Insertar un elemento es una operación $O(1)$ porque solo hay que añadirlo al principio de la segunda lista. Por ejemplo, añadir 6 daría `[1, 2, 3]` y `[6, 5, 4]`. Eliminar un elemento es más complejo, porque hay 2 situaciones:

- La primera eliminación quitaría el primer elemento de la primera lista ($O(1)$), y quedarían como `[2, 3]` y `[6, 5, 4]`. Las dos siguientes eliminaciones también serían $O(1)$, y las dos listas quedarían como `[]` y `[6, 5, 4]`.
- La siguiente eliminación no puede usar la primera lista porque está vacía. El siguiente elemento está al final de la segunda. En esta situación se le da la vuelta a la segunda lista, y se cambia por la primera. Es decir, se pasa de `[]` y `[6, 5, 4]` a `[4, 5, 6]` y `[]`. Ahora se puede hacer la eliminación nuevamente quitando el primer elemento de la primera lista. Esta operación tiene complejidad $O(n)$, pero garantiza que las siguientes n eliminaciones son $O(1)$ (complejidad algorítmica amortizada).

El API debe ser el siguiente:

- `queue:empty()`, devuelve una cola vacía.
- `queue:insert(Queue, Elem)`, inserta el elemento y devuelve la cola resultante.
- `queue:remove(Queue)`, elimina el elemento al frente. Devuelve `{ok, Elem, Queue}` o el átomo `empty` si la cola está vacía.

Ejercicio 2 (Procesos en línea)

Implemente un módulo `line` que cree una secuencia de procesos, donde cada proceso conoce el PID del siguiente en la secuencia. El módulo debe proporcionar las siguientes funciones:

- `line:start(N)`: crea N procesos y devuelve el PID del primero.
- `line:send(Pid, Msg)`: transmite `Msg` por la secuencia. Cada proceso imprime su número y el mensaje, y lo pasa al siguiente.
- `line:bounce(Pid, Msg, Times)`: el mensaje se transmite hacia el final decrementando `Times`. Si al llegar al final `Times > 0`, rebota hacia atrás y adelante iterativamente.
- `line:stop(Pid)`: detiene todos los procesos de la secuencia.

Ejercicio 3 (Anillo de procesos)

Implemente un módulo `ring` que cree un anillo de procesos, donde cada proceso conoce el PID del siguiente. Proporcionará:

- `ring:start(N)`: crea N procesos en anillo y devuelve el PID del primero.
- `ring:send(Pid, N, Msg)`: procesa `Msg` a lo largo de N pasos (puede dar múltiples vueltas al anillo).
- `ring:stop(Pid)`: detiene todos los procesos del anillo.

Bloque II: Paralelismo

Práctica 1: Cuenta las Apariciones de una Letra

Asignatura: Concurrency and Parallelism (Bloque II) **Puntuación:** 0.25 pts **Fecha Límite:** 20–22 de abril de 2026

Dada una implementación secuencial en C que cuenta el número de apariciones de un carácter L en un vector de caracteres de tamaño n , se pide realizar su paralelización SPMD utilizando la librería MPI.

Código Secuencial de Referencia

```
int i, n, count = 0;
char *cadena;
char L;

n = atoi(argv[1]);
L = *argv[2];
cadena = (char *) malloc(n * sizeof(char));
inicializaCadena(cadena, n);

for (i = 0; i < n; i++) {
    if (cadena[i] == L) {
        count++;
    }
}
printf("El numero de apariciones de la letra %c es %d", L, count);
free(cadena);
```

Requisitos de Paralelización

- Implementación según el modelo **SPMD** (Single Program, Multiple Data).
- La entrada/salida (lectura de argumentos de entrada y `printf` final) debe realizarla exclusivamente el **proceso 0**.
- Distribuir los parámetros n y L a todos los procesos mediante llamadas punto a punto (`MPI_Send` / `MPI_Recv`).
- Reparto de la carga de trabajo en el bucle `for` mediante un "paso" intercalado: `i += numprocs` en lugar de `i++`.

- Recoger el número de apariciones detectado por cada proceso en el proceso 0 usando comunicaciones punto a punto (MPI_Send / MPI_Recv).

Práctica 2: Colectivas MPI en la Cuenta de Apariciones

Asignatura: Concurrency and Parallelism (Bloque II) **Puntuación:** 0.50 pts **Fecha Límite:** 20–22 de abril de 2026

Partiendo de la paralelización realizada en la Práctica 1 del Bloque II, se aplican mejoras sustituyendo las comunicaciones punto a punto por operaciones colectivas de MPI y por implementaciones propias.

Uso y Desarrollo de Colectivas MPI

- Inicialmente, sustituir el envío y recolección manual por operaciones colectivas estándar de MPI (como MPI_Bcast y MPI_Reduce).
- **Distribución con Árbol Binomial (MPI_BinomialBcast):** Diseñar e implementar una función propia de difusión basada en una estructura de árbol binomial para distribuir n y L .
 - Asumir por simplicidad que el proceso raíz (*root*) es el 0.
 - En el paso i , los procesos con $myrank < 2^{i-1}$ envían el mensaje al proceso $myrank + 2^{i-1}$.
 - Paso 1: $0 \rightarrow 1$; Paso 2: $0 \rightarrow 2, 1 \rightarrow 3$; Paso 3: $0 \rightarrow 4, 1 \rightarrow 5, 2 \rightarrow 6, 3 \rightarrow 7$.
 - Mantener los mismos parámetros y control de errores que la función estándar MPI_Bcast.
- **Recolección con Árbol Plano (MPI_FlatticeCollective):** Diseñar e implementar una colectiva propia para la recolección del conteo total (operación suma de enteros).
 - Inicialmente utilizar las mismas operaciones MPI_Send / MPI_Recv (bucle for de recepciones en proceso 0).
 - Mantener la cabecera y parámetros equivalentes a las colectivas estándar de MPI.

Práctica 3: Descomposición de Dominio — Vector de Similitud de ADN

Asignatura: Concurrency and Parallelism (Bloque II) **Puntuación:** 0.75 pts **Fecha Límite:** 4–6 de mayo de 2026

El objetivo es calcular la medida de similitud entre dos conjuntos de secuencias de ADN. Se dispone de M secuencias de texto de N bases cada una (caracteres A, C, G, T, N). El resultado es un vector de M elementos donde el elemento i representa la similitud entre la secuencia i del primer conjunto y la secuencia i del segundo.

Código Secuencial Base (similarity.c)

```
int main(int argc, char *argv[]) {
    int *data1, *data2, *result;
    data1 = (int *) malloc(M * N * sizeof(int));
    data2 = (int *) malloc(M * N * sizeof(int));
    result = (int *) malloc(M * sizeof(int));

    /* Initialize Matrices & Compute */
}
```

```

for (i = 0; i < M; i++) {
    result[i] = 0;
    for (j = 0; j < N; j++) {
        result[i] += base_distance(data1[i * N + j], data2[i * N + j]);
    }
}

free(data1); free(data2); free(result);
}

```

Descomposición de Dominio y Paralelización

- **Descomposición de Dominio:** Dividir las matrices entre p procesos, correspondiendo $rows = M / p$ filas a cada proceso. Inicialmente asumir $M \bmod p = 0$, pero en la entrega final debe contemplarse cualquier número de procesos.
- **Operaciones Colectivas:** La inicialización de la matriz y la entrada/salida (`printf`) la realiza el proceso 0. La distribución de los datos de entrada y la recolección del vector resultado debe realizarse obligatoriamente mediante operaciones colectivas de MPI.
- **Medición de Tiempos:** Medir e imprimir por separado el tiempo de comunicaciones y el tiempo de computación de cada proceso. Para NP procesos, se deben mostrar exactamente **2 veces NP** tiempos.